

Informe Técnico: FNBC y Desnormalización Controlada («Food Store»)

Parte 1: Análisis y aplicación de la FNBC (ControlLoteAlmacen)

a) Dependencias Funcionales (DF)

A partir del relevamiento de la regla de negocio del área de logística, se identifican las siguientes dependencias funcionales formales:

- DF1: LotelID, DepositoID -> ResponsableControlID (Para un lote y un depósito dados, el responsable de control es único).
- DF2: ResponsableControlID -> DepositoID (Cada responsable pertenece y controla desde un único depósito).

b) Clausuras y Claves Candidatas

- Clausura de {LotelID, DepositoID}+: {LotelID, DepositoID, ResponsableControlID} => Es clave candidata.
- Clausura de {LotelID, ResponsableControlID}+: {LotelID, ResponsableControlID, DepositoID} (aplicando DF2) => Es clave candidata.
- **Atributos primos:** Todos los atributos del esquema (LotelID, DepositoID, ResponsableControlID), ya que forman parte de al menos una clave candidata. No existen atributos no primos.

c) Evaluación de la Forma Normal de Boyce-Codd (FNBC)

El esquema inicial no cumple con la FNBC. Si bien se encontraba admitido bajo la Tercera Forma Normal (3FN) por una excepción sobre atributos primos, la FNBC exige estrictamente que todo determinante sea una superclave. En este caso, la dependencia DF2 (ResponsableControlID -> DepositoID) viola la FNBC dado que su determinante (ResponsableControlID) no es una superclave del esquema (no permite identificar por sí solo el lote).

d) Anomalías en la instancia de ejemplo

Utilizando los datos provistos en el dominio, la violación detectada habilita las siguientes anomalías operativas:

- **Insertión:** No es posible registrar un nuevo responsable de control en el sistema (asociado a su respectivo depósito) si todavía no se le ha asignado ningún lote para controlar, debido a que lote_id forma parte de la clave primaria y no admite valores nulos.

- **Borrado:** Si se elimina por completo el registro del lote 503, se pierde de forma accidental la información maestra de que el responsable 802 pertenece al depósito 31.
- **Actualización:** Si el responsable 801 es trasladado a otro depósito, se deben actualizar manualmente múltiples filas (correspondientes a los lotes 501 y 502); olvidar alguna de ellas genera una inconsistencia grave en los datos.

e y f) Descomposición sin pérdida y migración

Aplicando el algoritmo de descomposición visto en clase (reflejado en el script `tp_fnbc_control_lote.sql`), la relación se dividió en dos tablas independientes: `responsable_deposito` y `lote_responsable`.

- **Justificación de la unión sin pérdida:** La reunión natural (JOIN) entre ambas tablas es completamente sin pérdida de información porque el atributo común utilizado como vínculo (`responsable_control_id`) actúa como superclave (Primary Key) en la tabla resultante de datos maestros `responsable_deposito`.

Parte 2: Consulta con Desnormalización Controlada

a) EXPLAIN ANALYZE inicial (Antes de la optimización)

Se adjunta la captura de pantalla obtenida en pgAdmin ejecutando la consulta original pesada con múltiples uniones (JOINS).

The screenshot shows the pgAdmin 4 interface with the following details:

- Query:** `EXPLAIN ANALYZE SELECT c.nombre AS categoria, (sum((dp.cantidad::numeric * dp.precio_unitario))) DESC`
- Query Plan:**
 - 1 Limit (cost=8.29..8.29 rows=1 width=250) (actual time=0.355..0.356 rows=0 loops=1)
 - 2 Sort (cost=8.29..8.29 rows=1 width=250) (actual time=0.353..0.354 rows=0 loops=1)
 - 3 Sort Key: (sum((dp.cantidad::numeric * dp.precio_unitario))) DESC
 - 4 Sort Method: quicksort Memory: 25kB
 - 5 GroupAggregate (cost=8.25..8.28 rows=1 width=250) (actual time=0.192..0.193 rows=0 loops=1)
 - 6 Group Key: c.nombre
 - 7 Sort (cost=8.25..8.25 rows=1 width=238) (actual time=0.191..0.192 rows=0 loops=1)
 - 8 Sort Key: c.nombre
 - 9 Sort Method: quicksort Memory: 25kB
 - 10 Nested Loop (cost=0.15..8.24 rows=1 width=238) (actual time=0.014..0.014 rows=0 loops=1)
 - 11 Join Filter: (dp.nro_pedido = ped.nro_pedido)
 - 12 Nested Loop (cost=0.15..8.23 rows=1 width=246) (actual time=0.013..0.014 rows=0 loops=1)
 - 13 Nested Loop (cost=0.00..0.01 rows=1 width=36) (actual time=0.013..0.013 rows=0 loops=1)
 - 14 Join Filter: (dp.id_producto = pr.id_producto)
 - 15 Seq Scan on detalle_pedido dp (cost=0.00..0.00 rows=1 width=36) (actual time=0.012..0.013 rows=0 loops=1)
 - 16 Seq Scan on producto pr (cost=0.00..0.00 rows=1 width=16) (never executed)
 - 17 Index Scan using categoria_pkey on categoria c (cost=0.15..8.17 rows=1 width=226) (never executed)
 - 18 Index Cond: (id_categoria = pr.id_categoria)
 - 19 Seq Scan on pedido ped (cost=0.00..0.00 rows=1 width=8) (never executed)
 - 20 Filter: ((fecha::date = CURRENT_DATE))
 - 21 Planning Time: 5.442 ms
 - 22 Execution Time: 0.776 ms
- Total rows:** 22
- Query complete:** 00:00:00.043

Diagnóstico: El plan de ejecución muestra un costo elevado dominado por un escaneo secuencial (Seq Scan) sobre la tabla transaccional y sucesivas uniones de tablas bajo alta exigencia.

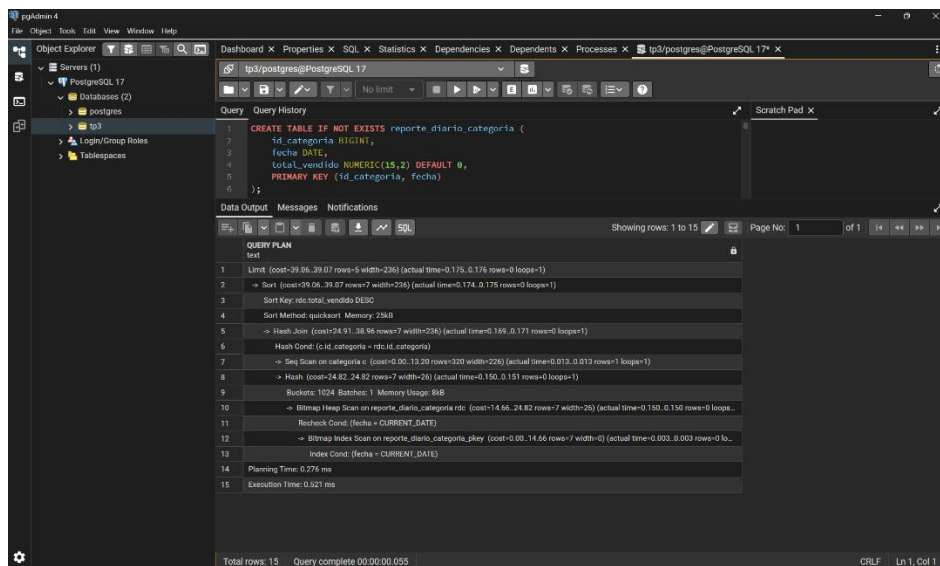
b) Elección del patrón de desnormalización

Se optó por implementar una **columna precalculada sincronizada mediante un disparador transaccional (trigger)**.

- **Evidencia que lo motiva:** El alto costo computacional y el tiempo de respuesta del EXPLAIN ANALYZE inicial al cruzar cuatro tablas de gran volumen en tiempo real.
- **Mecanismo de sincronización:** Un disparador transaccional en PL/pgSQL intercepta cada inserción, recalcula los montos y actualiza la tabla de resumen dentro de la misma transacción blindada, haciendo imposible el desfase.
- **Reversibilidad:** Es totalmente reversible sin pérdida de información, ya que la fuente de verdad original (detalle_pedido y pedido) permanece intacta.

c y d) EXPLAIN ANALYZE optimizado (Después de la desnormalización)

Se adjunta la captura de pantalla obtenida ejecutando la consulta simplificada sobre la estructura desnormalizada reporte_diario_categoria.



e) Auditoría de sincronización

Se ejecutó el script de auditoría programado para verificar la integridad del dato redundante, el cual arrojó un conjunto de resultados **vacío**, comprobando fehacientemente que el disparador mantiene la sincronización perfecta frente a la fuente de verdad.

Repositorio de código fuente (GitHub):

<https://github.com/santinodallatorre2501-bit/Resolucion-TPUnidad4.git>